Hake_variables

Setup

```
# Check current working directory
devtools::load_all(here::here())
```

```
i Loading mom6-based-groundfish-indices
Registered S3 method overwritten by 'httr':
  method           from
  print.cache_info hoardr
```

```
library(dplyr)
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(tidyr)
library(corrplot)
```

```
corrplot 0.95 loaded
```

```
library(rerddap)
library(ggplot2)
library(rerddap)
library(ggplot2)
library(lubridate)
```

```
Attaching package: 'lubridate'

The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

```
library(rerddap)
library(purrr)
library(stringr)
library(reticulate)
```

Next, we need to set up our python environment for reticulate and make sure it is calling the correct version of icechunk and arraylake to be able to interact with Eartmover. Reticulate automatically loads an environment when you start a new session and ensuring the python environment you are using with reticulate is correctly specifying the correct version of python and the Earthmover associated packages is key.

```
# Force reticulate to pull down the newest package versions
#reticulate::py_install("dask")
py_require("icechunk>=2.0.0")
py_require("arraylake>=1.0.0")
py_require("zarr")
py_require("h5netcdf")
py_require("h5py")
py_require("xarray")
py_require("scipy")
py_require("dask")
# Import the fresh libraries
arraylake <- import("arraylake") #the data visual layer of Earthmover
zarr <- import("zarr") #interact with compressed data chunks and JSON metadata
xr   <- import("xarray") #python library for working with arrays
np   <- np <- import("numpy", convert = FALSE) #necessary for the NaN checking block
pd   <- import("pandas")
time <- import("time")
```

```
dask <- import("dask")

# Verify the version
print(arraylake$`__version__`)
```

```
[1] "1.7.0"
```

Now we will load the data from Earthmover and a static .netcdf. Whether or not this loads correctly will identify whether the python environment is loaded correctly in R and may require trounblshooting

```
# Connect to your NOAA data repository
client <- arraylake$Client()
#token issue? go to terminal arraylake auth login
repo <- client$get_repo("NOAA-PMEL/cefi-nep-hindcast-monthly")
print(repo)
```

```
<icechunk.Repository (v2)>
storage:
    <icechunk.Storage>
    type: S3 (native)
    bucket: ems-noaa-pmel-bad60303
    prefix: b8fvrheq_cefi_nep_hindcast_monthly
    region: us-east-1
config: <RepositoryConfig ...>
```

```
#If this runs then Earthmover is functioning!

ds_depth = xr$open_dataset("../STATIC/ocean_static.deptho.nc")

#If this runs then xarray has appropriate dependencies loaded
```

```
# assign months, latitudes, and depths to filter
# all are integers, depths are in meters
# filters are inclusive (closed interval), e.g. depth 0-180 includes depths 0 and 180 and ev

lifestage_dict <- list(
  prec = list( # spawn
    min_month = 4,
    max_month = 10,
```

```
    min_lat   = 42,
    max_lat   = 47,
    min_depth = 50,
    max_depth = 350,
    min_iso = 100,
    max_iso=2000
  ),
   spawn = list( # spawn
    min_month = 1,
    max_month = 3,
    min_lat   = 31,
    max_lat   = 36,
    min_depth = 130,
    max_depth = 500,
    min_iso = 100,
    max_iso=2000
  ), #long extent shelf break
  egg = list( # egg
    min_month = 1,
    max_month = 3,
    min_lat   = 40,
    max_lat   = 60,
    min_depth = 40,
    max_depth = 60,
    min_iso = 100,
    max_iso=2000,
    max_dist = 30,
    min_dist = 0

  ), #long extent shelf break
  yolk = list( # yolk
    min_month = 1,
    max_month = 4,
    min_lat   = 31,
    max_lat   = 36,
    min_depth = 50,
    max_depth = 100,
    min_iso = 100,
    max_iso=2000
  ), #long extent shelf break
  JAc = list( # JAc
    min_month = 1,
```

```
    max_month = 4,
    min_lat   = 34.5,
    max_lat   = 42.5,
    min_depth = 30,
    max_depth = 130,
    max_dist = 30,
    min_dist = 0
  ),
    JAs = list( # JAs
    min_month = 1,
    max_month = 4,
    min_lat   = 31,
    max_lat   = 34.5,
    min_depth = 30,
    max_depth = 130,
    max_dist = 30,
    min_dist = 0
  ),
    MSc = list( # MSc
    min_month = 5,
    max_month = 9,
    min_lat   = 34.5,
    max_lat   = 42.5,
    min_depth = 30,
    max_depth = 130,
    max_dist = 30,
    min_dist = 0
  ),
    MSs = list( # MSs
    min_month = 5,
    max_month = 9,
    min_lat   = 31,
    max_lat   = 34.5,
    min_depth = 30,
    max_depth = 130,
    max_dist = 30,
    min_dist = 0
  ),
    MSn = list( # MSn
    min_month = 5,
    max_month = 9,
    min_lat   = 42.5,
```

```
    max_lat   = 47,
    min_depth = 30,
    max_depth = 130,
    max_dist = 30,
    min_dist = 0
  ),
    larv = list( # larv
    min_month = 2,
    max_month = 5,
    min_lat   = 31,
    max_lat   = 36,
    min_depth = 50,
    max_depth = 200,
    min_iso = 100,
    max_iso=2000
  ),
  latelarv = list( # latelarv
    min_month = 3,
    max_month = 6,
    min_lat   = 31,
    max_lat   = 37,
    min_depth = 50,
    max_depth = 300,
    min_iso = 100,
    max_iso=2000
  ),
  late = list( # late
    min_month = 3,
    max_month = 6,
    min_lat   = 33.5,
    max_lat   = 34.5,
    min_depth = 75,
    max_depth = 275
  ),
    JA = list( # Jan to april
    min_month = 1,
    max_month = 4
  ),
  AS = list( # April to september
    min_month = 4,
    max_month = 9
  )
```

```
)
```

```
<xarray.Dataset> Size: 2MB
Dimensions:  (lat: 815, lon: 341)
Coordinates:
  * lat      (lat) float64 7kB 10.81 10.89 10.98 11.07 ... 80.55 80.63 80.72
  * lon      (lon) float64 3kB 156.9 157.2 157.5 157.8 ... 254.4 254.7 255.0
Data variables:
    dist     (lat, lon) float64 2MB ...
Attributes:
    Description:    Distance to shore of grid points from MOM6-NEP regridded ...
    Caution:        Do not use for Alaska waters. Distances are accurate only...
    Creation date:  19-May-2025 11:24:10

---MLDegg---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.13 minutes

---MLDlatelarv---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.07 minutes

---MLDlateyolk---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.05 minutes
```

```
---MLDlateyolk---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.03 minutes

---SSHjac---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.05 minutes

---SSHjas---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.04 minutes

---SSHmsc---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.03 minutes

---SSHmss---
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial dimensions natively...
```

```
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Runtime: 0.04 minutes

---TEMPpre---
Slicing spatial boundaries...
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial/depth dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Applying 1-year environmental precondition lag (Years + 1)...
Runtime: 1.52 minutes

---TEMPspawn---
Slicing spatial boundaries...
Standardizing mask coordinate names...
Interpolating mask grid onto variable layout...
Applying mask layer...
Collapsing spatial/depth dimensions natively...
Extracting values safe from memory overhead...
Aggregating annual timeline numbers in R...
Applying 1-year environmental precondition lag (Years + 1)...
Runtime: 0.75 minutes

# A tibble: 6 x 29
   year NPGOprec NPGOspawn NPGOegg NPGOyolk NPGOJAc NPGOJAs NPGOMSc NPGOMSs
  <dbl>    <dbl>     <dbl>   <dbl>    <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1  1950  -0.550    -1.53   -1.53    -1.36   -1.36   -1.36   -0.622  -0.622
2  1951  -1.12     -0.487  -0.487   -0.506  -0.506  -0.506  -1.24   -1.24
3  1952  -0.729    -1.03   -1.03    -0.882  -0.882  -0.882  -0.883  -0.883
4  1953   0.944     0.970   0.970    1.04    1.04    1.04    0.856   0.856
5  1954  -0.0863    0.777   0.777    0.489   0.489   0.489  -0.0830 -0.0830
6  1955  -0.0231   -0.0463 -0.0463  -0.0246 -0.0246 -0.0246  0.0644  0.0644
# i 20 more variables: NPGOMSn <dbl>, NPGOlarv <dbl>, NPGOlatelarv <dbl>,
#   NPGOlate <dbl>, NPGOJA <dbl>, NPGOAS <dbl>, PDOprec <dbl>, PDOspawn <dbl>,
#   PDOegg <dbl>, PDOyolk <dbl>, PDOJAc <dbl>, PDOJAs <dbl>, PDOMSc <dbl>,
#   PDOMSs <dbl>, PDOMSn <dbl>, PDOlarv <dbl>, PDOlatelarv <dbl>,
#   PDOlate <dbl>, PDOJA <dbl>, PDOAS <dbl>

Joining with `by = join_by(year, NPGOJAs, PDOJAs)`
```